Daily Challenge :

Preprocess & fine-tune transformer-based models

What you will create?

By the end of this challenge, you will have a fine-tuned transformer model (BERT or XLM-RoBERTa) capable of classifying text into different categories. Additionally, you will structure the data for training, validate it using cross-validation, and understand how to optimize these models for better performance.

Task 1:

Understanding BERT and XLM-RoBERTa

Objective: Learn how transformer models work and their role in NLP tasks.

Instructions:

- Read through the descriptions of BERT and XLM-RoBERTa.
- Understand how these models process text using tokenization.
- Learn about different pre-trained versions of these models and their characteristics.

BERT (Bidirectional Encoder Representations from Transformers): It reads text in both directions simultaneously to understand context. However, standard BERT is primarily trained on English.

XLM-RoBERTa: This is a multilingual version of RoBERTa. Since your dataset includes French, Thai, Arabic, Urdu, and more, XLM-RoBERTa is actually the superior choice here because it was pre-trained on 100 different languages.

Understand how these models process text using tokenization

In BERT-style models, text isn't just split into words; it's split into sub-words using a method called Byte-Pair Encoding or WordPiece. This prevents "Out of Vocabulary" errors.

```
!pip install transformers[torch] datasets
```

```
Requirement already satisfied: transformers[torch] in /usr/local/lib/python3.12/dist-packages (5.0.0)
Requirement already satisfied: filelock in /usr/local/lib/python3.12/dist-packages (from transformers[torch]) (
Requirement already satisfied: huggingface-hub<2.0,>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from tra
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3.12/dist-packages (from transformers[torch]
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from transformers[to
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3.12/dist-packages (from transformers[torch]
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/python3.12/dist-packages (from transformers[
Requirement already satisfied: tokenizers<=0.23.0,>=0.22.0 in /usr/local/lib/python3.12/dist-packages (from tra
Requirement already satisfied: typer-slim in /usr/local/lib/python3.12/dist-packages (from transformers[torch])
Requirement already satisfied: safetensors>=0.4.3 in /usr/local/lib/python3.12/dist-packages (from transformers
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.12/dist-packages (from transformers[torch])
Requirement already satisfied: torch>=2.2 in /usr/local/lib/python3.12/dist-packages (from transformers[torch])
Requirement already satisfied: accelerate>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from transformers[
Requirement already satisfied: pyarrow>=15.0.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (18.1
Requirement already satisfied: dill<0.3.9,>=0.3.0 in /usr/local/lib/python3.12/dist-packages (from datasets) (0
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (from datasets) (2.2.2)
Requirement already satisfied: requests>=2.32.2 in /usr/local/lib/python3.12/dist-packages (from datasets) (2.3
Requirement already satisfied: xxhash in /usr/local/lib/python3.12/dist-packages (from datasets) (3.6.0)
Requirement already satisfied: multiprocess<0.70.17 in /usr/local/lib/python3.12/dist-packages (from datasets)
Requirement already satisfied: fsspec<=2025.3.0,>=2023.1.0 in /usr/local/lib/python3.12/dist-packages (from fss
Requirement already satisfied: psutil in /usr/local/lib/python3.12/dist-packages (from accelerate>=1.1.0->trans
Requirement already satisfied: aiohttp!=4.0.0a0,!=4.0.0a1 in /usr/local/lib/python3.12/dist-packages (from fsspe
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub
Requirement already satisfied: typer in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=1.3
Requirement already satisfied: typing-extensions>=4.1.0 in /usr/local/lib/python3.12/dist-packages (from huggin
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from reques
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.32.2->
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from torch>=2.2->transfor
Requirement already satisfied: sympy>=1.13.3 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2->trans
Requirement already satisfied: networkx>=2.5.1 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2->tra
Requirement already satisfied: jinja2 in /usr/local/lib/python3.12/dist-packages (from torch>=2.2->transformers
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas->
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas->datasets
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp
Requirement already satisfied: aiosignal>=1.4.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0
Requirement already satisfied: attrs>=17.3.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a0,
```

```
Requirement already satisfied: multidict<7.0,>=4.5 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0
Requirement already satisfied: propcache>=0.2.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0a
Requirement already satisfied: yarl<2.0,>=1.17.0 in /usr/local/lib/python3.12/dist-packages (from aiohttp!=4.0.0
Requirement already satisfied: anyio in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->hugging
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from sympy>=1.13.
Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggi
Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer->huggingface
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from typer->hug
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from jinja2->torch>=
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=1
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.
```

```
import pandas as pd
from transformers import BertTokenizer, XLMRobertaTokenizer

df = pd.read_csv('train.csv')

bert_tokenizer = BertTokenizer.from_pretrained('bert-base-multilingual-cased')
xlm_tokenizer = XLMRobertaTokenizer.from_pretrained('xlm-roberta-base')

sample_premise = df.iloc[0]['premise']
sample_hypothesis = df.iloc[0]['hypothesis']

print(f"Original Premise: {sample_premise}")
print(f"Original Hypothesis: {sample_hypothesis}\n")

#BERT Tokenization
bert_input = bert_tokenizer(sample_premise, sample_hypothesis)
print("--- BERT Tokens ---")
print(bert_tokenizer.convert_ids_to_tokens(bert_input['input_ids']))

#XLM-RoBERTa Tokenization
xlm_input = xlm_tokenizer(sample_premise, sample_hypothesis)
print("\n--- XLM-RoBERTa Tokens ---")
print(xlm_tokenizer.convert_ids_to_tokens(xlm_input['input_ids']))
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/t
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limi
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a
tokenizer_config.json: 100%  49.0/49.0 [00:00<00:00, 4.24kB/s]
vocab.txt:  996k/? [00:00<00:00, 14.4MB/s]
tokenizer.json:  1.96M/? [00:00<00:00, 19.5MB/s]
tokenizer_config.json: 100%  25.0/25.0 [00:00<00:00, 2.42kB/s]
sentencepiece.bpe.model: 100%  5.07M/5.07M [00:00<00:00, 37.7MB/s]
tokenizer.json:  9.10M/? [00:00<00:00, 32.0MB/s]
Original Premise: and these comments were considered in formulating the interim rules.
Original Hypothesis: The rules developed in the interim were put together with these comments in mind.

--- BERT Tokens ---
['[CLS]', 'and', 'these', 'comments', 'were', 'considered', 'in', 'formula', '##ting', 'the', 'interim', 'rules',

--- XLM-RoBERTa Tokens ---
['<s>', '_and', '_these', '_comments', '_were', '_considered', '_in', '_formula', 'ting', '_the', '_inter', 'im',
```

Task 2:

Tokenizing Text

Objective: Understand how to tokenize text using pre-trained tokenizers.

Instructions:

- Use the BertTokenizer and XLMRobertaTokenizer to convert sentences into tokenized input.
- Explore the different token types, such as input_ids, attention_mask, and labels.
- Experiment with single-sentence and two-sentence tokenization.

```
import torch
from transformers import AutoTokenizer
```

```
model_name = 'bert-base-multilingual-cased'
tokenizer = AutoTokenizer.from_pretrained(model_name)

premise = "The rules were put together with these comments in mind."
hypothesis = "The rules were developed recently."

encoded_dict = tokenizer(
    text=premise,
    text_pair=hypothesis,
    add_special_tokens=True,
    max_length=32,
    padding='max_length',
    truncation=True,
    return_attention_mask=True,
    return_tensors='pt'
)

# Results:
print("Successfully created keys:", encoded_dict.keys())

print("\n--- 1. Input IDs ---")
print(encoded_dict['input_ids'])

print("\n--- 2. Attention Mask ---")
print(encoded_dict['attention_mask'])

print("\n--- 3. Decoded Sequence ---")
print(tokenizer.decode(encoded_dict['input_ids'][0]))
```

```
Successfully created keys: KeysView({'input_ids': tensor([[  101, 10117, 23123, 10309, 14499, 14229, 10169, 11762
         21133,   119,   102, 10117, 23123, 10309, 14628, 23746,   119,   102,
             0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
             0,     0]]), 'token_type_ids': tensor([[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1,
         0, 0, 0, 0, 0, 0, 0, 0]]), 'attention_mask': tensor([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
         0, 0, 0, 0, 0, 0, 0, 0]])})

--- 1. Input IDs ---
tensor([[  101, 10117, 23123, 10309, 14499, 14229, 10169, 11762, 61565, 10106,
         21133,   119,   102, 10117, 23123, 10309, 14628, 23746,   119,   102,
             0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
             0,     0]])

--- 2. Attention Mask ---
tensor([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0]])

--- 3. Decoded Sequence ---
[CLS] The rules were put together with these comments in mind. [SEP] The rules were developed recently. [SEP] [PA
```

Task 3:

Preparing Input Data for the Model Objective: Format input data correctly for transformer models.

Instructions:

- Ensure that input sentences are padded and possibly truncated to max_length.
- Understand and set special tokens such as ~~and~~.
- Learn about attention_mask and how it helps the model ignore padding tokens.

Functions to use:

tokenizer.encode_plus() tokenizer.special_tokens_map tokenizer.vocab_size

```
print(f"Special Tokens Map: {tokenizer.special_tokens_map}")
print(f"Vocabulary Size: {tokenizer.vocab_size}")

# To check for <s> and </s> :
print(f"CLS token: {tokenizer.cls_token} (ID: {tokenizer.cls_token_id})")
print(f"SEP token: {tokenizer.sep_token} (ID: {tokenizer.sep_token_id})")
```

```
Special Tokens Map: {'unk_token': '[UNK]', 'sep_token': '[SEP]', 'pad_token': '[PAD]', 'cls_token': '[CLS]', 'mas
Vocabulary Size: 119547
CLS token: [CLS] (ID: 101)
SEP token: [SEP] (ID: 102)
```

```
import torch
from torch.utils.data import Dataset

class NliDataset(Dataset):
    def __init__(self, df, tokenizer, max_len=128):
```

```
        self.df = df
        self.tokenizer = tokenizer
        self.max_len = max_len

    def __len__(self):
        return len(self.df)

    def __getitem__(self, index):
        premise = str(self.df.iloc[index]['premise'])
        hypothesis = str(self.df.iloc[index]['hypothesis'])
        label = self.df.iloc[index]['label']

        encoding = self.tokenizer(
            premise,
            hypothesis,
            add_special_tokens=True,
            max_length=self.max_len,
            padding='max_length',
            truncation=True,
            return_attention_mask=True,
            return_tensors='pt'
        )

        return {
            'input_ids': encoding['input_ids'].flatten(),
            'attention_mask': encoding['attention_mask'].flatten(),
            'labels': torch.tensor(label, dtype=torch.long)
        }

train_dataset = NliDataset(df, tokenizer)

# Test:
sample_item = train_dataset[0]
print("Keys in one data item:", sample_item.keys())
print("Shape of input_ids:", sample_item['input_ids'].shape)
```

```
Keys in one data item: dict_keys(['input_ids', 'attention_mask', 'labels'])
Shape of input_ids: torch.Size([128])
```

```
# Final check for Task 3 requirements:
print("--- Task 3 Metadata Check ---")

print(f"Special Tokens Map: {tokenizer.special_tokens_map}")

# Vocabulary Size:
print(f"Tokenizer Vocab Size: {tokenizer.vocab_size}")

# Attention Mask logic:
sample_item = train_dataset[0]
print("\n--- Attention Mask Verification ---")
print(f"Input IDs (first 15): {sample_item['input_ids'][:15]}")
print(f"Mask IDs (first 15):  {sample_item['attention_mask'][:15]}")
```

```
--- Task 3 Metadata Check ---
Special Tokens Map: {'unk_token': '[UNK]', 'sep_token': '[SEP]', 'pad_token': '[PAD]', 'cls_token': '[CLS]', 'mas
Tokenizer Vocab Size: 119547

--- Attention Mask Verification ---
Input IDs (first 15): tensor([  101, 10111, 11762, 61565, 10309, 14289, 10106, 29659, 12141, 10105,
        63313, 23123,   119,   102, 10117])
Mask IDs (first 15):  tensor([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1])
```

Learn about attention_mask and how it helps the model ignore padding tokens:

- A '1' in the mask means the model processes that token.
- A '0' (usually at the end of the tensor) tells the model to ignore that padding.

Task 4:

Loading and Exploring the Dataset Objective: Load the dataset and explore its structure.

Instructions:

- Load the training and testing data from CSV files.
- Display the first few rows to understand its structure.
- Identify the columns needed for training the model.

Functions to use:

pd.read_csv() df.head() df.shape

```
import pandas as pd

df = pd.read_csv('train.csv')

print(f"Dataset Shape: {df.shape}")

print("\n--- First 5 Rows ---")
print(df.head())

# Columns needed for training:
cols_for_training = ['premise', 'hypothesis', 'label']
print(f"\nTarget Training Columns: {cols_for_training}")
```

```
Dataset Shape: (12120, 6)

--- First 5 Rows ---
           id                                           premise  \
0  5130fd2cb5  and these comments were considered in formulat...
1  5b72532a0b  These are issues that we wrestle with in pract...
2  3931fbe82a  Des petites choses comme celles-là font une di...
3  5622f0c60b  you know they can't really defend themselves l...
4  86aaa48b45  ในการเล่นบทบาทสมมุติก็เช่นกัน โอกาสที่จะได้แสด...

                                        hypothesis lang_abv language  label
0  The rules developed in the interim were put to...       en  English      0
1  Practice groups are not permitted to work on t...       en  English      2
2             J'essayais d'accomplir quelque chose.       fr   French      0
3  They can't defend themselves because of their ...       en  English      0
4    เด็กสามารถเห็นได้ว่าชาติพันธุ์แตกต่างกันอย่างไร       th     Thai      1

Target Training Columns: ['premise', 'hypothesis', 'label']
```

```
import pandas as pd

# Loading both files:
train_df = pd.read_csv('train.csv')
test_df = pd.read_csv('test.csv')

# Final Check: Display first rows and shapes:
print(f"Train Shape: {train_df.shape} | Test Shape: {test_df.shape}")
print("\n--- Train Columns ---")
print(train_df.columns.tolist())
print("\n--- Test Columns ---")
print(test_df.columns.tolist())
```

```
Train Shape: (12120, 6) | Test Shape: (5195, 5)

--- Train Columns ---
['id', 'premise', 'hypothesis', 'lang_abv', 'language', 'label']

--- Test Columns ---
['id', 'premise', 'hypothesis', 'lang_abv', 'language']
```

Task 5:

Creating Cross-Validation Folds Objective: Implement k-fold cross-validation for training.

Instructions:

- Use StratifiedKFold to create 5 training-validation splits.
- Ensure that each fold maintains the same label distribution.
- Store the training and validation splits in separate lists.

Functions to use:

from sklearn.model_selection import StratifiedKFold kf.split() StratifiedKFold(shuffle=True)

```
from sklearn.model_selection import StratifiedKFold

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

# Lists to store splits:
train_folds = []
val_folds = []

# Creating the splits:
for train_index, val_index in skf.split(train_df, train_df['label']):
    train_folds.append(train_df.iloc[train_index])
    val_folds.append(train_df.iloc[val_index])
```

```
# Task 5:
print(f"Number of folds created: {len(train_folds)}")
print(f"First fold training size: {len(train_folds[0])}")
print(f"First fold validation size: {len(val_folds[0])}")

print("\nLabel distribution in Validation Fold 1:")
print(val_folds[0]['label'].value_counts(normalize=True))
```

```
Number of folds created: 5
First fold training size: 9696
First fold validation size: 2424

Label distribution in Validation Fold 1:
label
0    0.344884
2    0.334983
1    0.320132
Name: proportion, dtype: float64
```

Conclusion & Insights:

This challenge demonstrates the end-to-end workflow of adapting 'state of the art' Transformer models for a multilingual Natural Language Inference (NLI) task. By comparing BERT and XLM-RoBERTa, we observed how tokenization strategies like WordPiece and SentencePiece handle linguistic nuances and special tokens ([SEP] vs) differently. The successful implementation of the NliDataset class highlights the importance of precise data formatting specifically padding, truncation, and attention masking to ensure that the model correctly prioritizes meaningful text over structural filler during the training process.

A key insight from the exploratory phase was the balanced distribution of the entailment, neutral, and contradiction labels. This balance, coupled with the use of StratifiedKFold, ensures that each training cycle is exposed to a representative slice of the ground truth, preventing the model from developing a bias toward any specific class. Using five folds allows for a robust evaluation of the model's stability, providing a more reliable estimate of performance than a simple train-test split, especially when dealing with the diverse linguistic patterns found in train.csv.

Ultimately, the choice of XLM-RoBERTa as the primary architecture is justified by the multilingual nature of the dataset. Unlike standard monolingual models, XLM-RoBERTa's cross-lingual pre-training allows it to share semantic knowledge across different languages, such as English, French, and Thai, within the same embedding space. This approach not only streamlines the training pipeline but also maximizes the model's ability to generalize across the various languages present in the test set, leading to more accurate and reliable text classification.

```
Start coding or generate with AI.
```